

InvBot: Asistente Conversacional de Soporte al Inversionista
Activo para una Casa de Bolsa

Abel Sánchez

Maestría en Inteligencia de Negocios y Ciencia de Datos

Inteligencia Artificial

Juan Simón Isidro

29 de agosto de 2026

Tabla de Contenidos

1. Introducción y justificación
2. Objetivos
3. Delimitación del entorno profesional
4. Diseño de la lógica conversacional
5. Arquitectura e integración de datos
6. Decisiones de arquitectura y alternativas evaluadas
7. Seguridad, pruebas y estrategia de despliegue
8. Entregables del proyecto
9. Conclusiones y reflexión final
10. Referencias
11. Anexos

1. Introducción y justificación

Las casas de bolsa y sociedades de intermediación bursátil reciben un volumen alto de consultas repetitivas por parte de sus clientes activos: estado de órdenes, saldos, tarifas, tiempos de retiro y reporte de incidencias. Gran parte de estas consultas no requiere criterio humano ni asesoría de inversión, sino acceso rápido a información estructurada y registro consistente de solicitudes. Un asistente conversacional permite atender estas consultas 24/7, reducir la carga del equipo de servicio al cliente y garantizar que ninguna solicitud se pierda, al quedar registrada automáticamente en una base de datos en tiempo real.

Este proyecto diseña e implementa **InvBot**, un asistente conversacional para una casa de bolsa ficticia (**Andes Valores S.A.**, con actividad en la Bolsa de Valores de Quito — BVQ — y la Bolsa de Valores de Guayaquil — BVG) orientado a inversionistas que ya poseen una cuenta de inversión activa.

2. Objetivos

Objetivo general

Diseñar, modelar y prototipar un asistente conversacional capaz de resolver consultas frecuentes de inversionistas activos y registrar automáticamente sus solicitudes e incidencias en una base de datos relacional en tiempo real.

Objetivos específicos

- Estructurar un flujo conversacional que cubra consultas de saldo, órdenes, tarifas, retiros/depósitos, dividendos y horarios de mercado.
- Implementar un sistema de captura y escalamiento de incidencias (tickets) con estados de seguimiento.

- Diseñar un modelo de base de datos relacional que soporte clientes, cuentas, órdenes, tickets e interacciones.
- Automatizar la transferencia de datos entre la plataforma conversacional y la base de datos mediante webhooks.

3. Delimitación del entorno profesional

3.1 Perfil del usuario objetivo

Atributo	Descripción
Rol	Inversionista retail activo con cuenta abierta en Andes Valores S.A.
Edad	28–55 años
Perfil de riesgo	Conservador, moderado o agresivo (ya definido en onboarding previo)
Canal habitual	Aplicación móvil / portal web de trading, WhatsApp Business, chat web
Necesidad principal	Respuestas inmediatas fuera del horario de atención telefónica (8:00–17:00)
Nivel técnico	Medio; familiarizado con conceptos bursátiles básicos (orden, custodia, dividendo)
Fricciones actuales	Espera telefónica larga, falta de trazabilidad de reclamos, desconocimiento de tarifas exactas

3.2 Categorización de FAQs principales

1. **Estado de cuenta y saldos:** saldo disponible para invertir, valor en custodia, movimientos recientes.
2. **Órdenes de compra/venta:** estado de una orden (pendiente/ejecutada/cancelada), tiempo estimado de ejecución, cómo cancelar una orden vigente.
3. **Tarifas y comisiones:** comisión por operación bursátil, comisión de custodia mensual, IVA aplicable, tarifas de transferencia.
4. **Retiros y depósitos de fondos:** tiempos de acreditación, requisitos, cuentas bancarias asociadas autorizadas.

5. **Dividendos y derechos:** fechas de pago, forma de acreditación, retenciones aplicables.
6. **Horarios y calendario de mercado:** horario de negociación BVQ/BVG, feriados bursátiles, suspensiones de mercado.
7. **Incidencias y soporte técnico:** orden no ejecutada, error en la plataforma, reclamo sobre un cobro, solicitud de corrección.
8. **Escalamiento a asesor humano:** cuando la consulta excede el alcance informativo del bot (asesoría de inversión, quejas formales).

3.3 Documentación técnica básica necesaria

- **Tabla de comisiones vigente** (por operación, por custodia, por transferencia, IVA).
- **Política de retiros y depósitos** (tiempos de acreditación, montos mínimos, medios de pago aceptados).
- **Catálogo de instrumentos** que el cliente puede operar (acciones, obligaciones, papel comercial, valores de renta fija).
- **SLA de tickets de soporte:** tiempo máximo de respuesta por tipo de incidencia.
- **Calendario bursátil oficial** (BVQ/BVG): días y horarios de negociación, feriados.
- **Reglamento interno de atención al cliente** y criterios de escalamiento a asesor humano.

4. Diseño de la lógica conversacional (Fase 2)

4.1 Diagrama de flujo

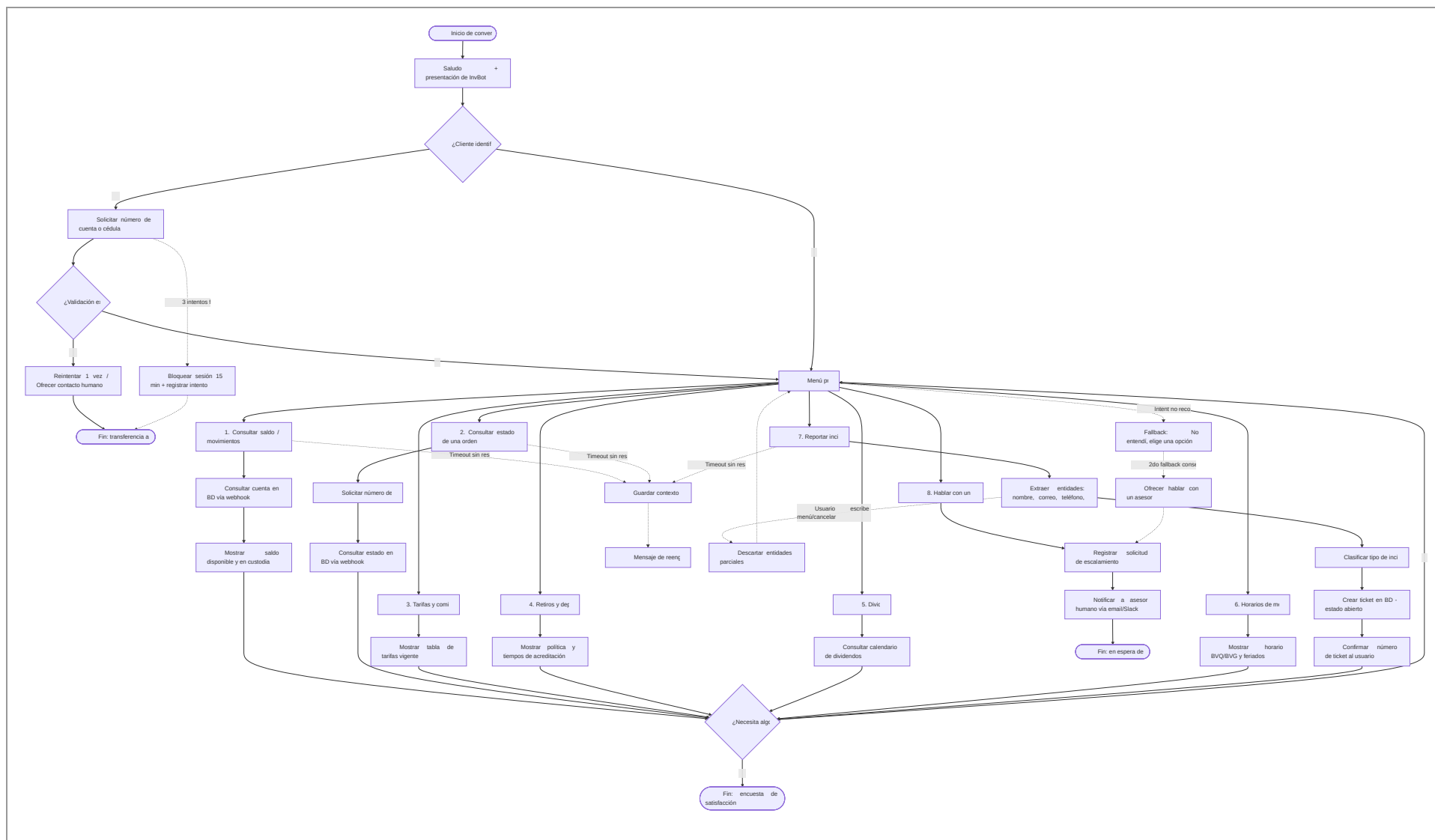


Figura 1. Árbol de decisión conversacional de InvBot, incluyendo manejo de casos límite (fallback, timeout, reintentos de autenticación).

flowchart TD

A([Inicio de conversación]) --> B[Saludo + presentación de InvBot]

B --> C{¿Cliente identificado?}

C -- No --> D[Solicitar número de cuenta o cédula]

D --> E{¿Validación exitosa?}

E -- No --> F[Reintentar 1 vez / Ofrecer contacto humano]

F --> Z1([Fin: transferencia a asesor])

E -- Sí --> G[Menú principal]

C -- Sí --> G[Menú principal]

G --> H1[1. Consultar saldo / movimientos]

G --> H2[2. Consultar estado de una orden]

G --> H3[3. Tarifas y comisiones]

G --> H4[4. Retiros y depósitos]

G --> H5[5. Dividendos]

G --> H6[6. Horarios de mercado]

G --> H7[7. Reportar incidencia]

G --> H8[8. Hablar con un asesor]

H1 --> Q1[Consultar cuenta en BD vía webhook]

Q1 --> R1[Mostrar saldo disponible y en custodia]

R1 --> M{¿Necesita algo más?}

H2 --> Q2[Solicitar número de orden]

Q2 --> R2[Consultar estado en BD vía webhook]

R2 --> M

H3 --> R3[Mostrar tabla de tarifas vigente]

R3 --> M

H4 --> R4[Mostrar política y tiempos de acreditación]

R4 --> M

H5 --> R5[Consultar calendario de dividendos]

R5 --> M

H6 --> R6[Mostrar horario BVQ/BVG y feriados]

R6 --> M

H7 --> I1[Extraer entidades: nombre, correo, teléfono, motivo]

I1 --> I2[Clasificar tipo de incidencia]

I2 --> I3[Crear ticket en BD - estado 'abierto']

I3 --> I4[Confirmar número de ticket al usuario]

I4 --> M

H8 --> J1[Registrar solicitud de escalamiento]

J1 --> J2[Notificar a asesor humano vía email/Slack]

J2 --> Z2([Fin: en espera de asesor])

M -- Sí --> G

M -- No --> Z3([Fin: encuesta de satisfacción])

```

%% Casos límite y manejo de excepciones
G -->|Intent no reconocido| X1[Fallback: 'No entendí, ¿puedes elegir una
opción del menú?']
X1 -->|2do fallback consecutivo| X2[Ofrecer 'hablar con un asesor']
X2 --> J1

H1 -->|Timeout de sesión sin respuesta| X3[Guardar contexto parcial]
H2 -->|Timeout de sesión sin respuesta| X3
H7 -->|Timeout de sesión sin respuesta| X3
X3 --> X4[Mensaje de reenganche: '¿Seguimos con tu consulta?']

I1 -->|Usuario escribe 'menú' o 'cancelar' a mitad de flujo| X5[Descartar
entidades parciales]
X5 --> G

D -->|3 intentos fallidos de autenticación| X6[Bloquear sesión 15 min +
registrar intento en interacciones_chatbot]
X6 --> Z1

```

Código fuente del diagrama en sintaxis Mermaid.js correspondiente a la Figura 1.

Casos límite cubiertos explícitamente:

Caso límite	Comportamiento definido
Intent fuera de alcance / no reconocido	Fallback conversacional; al segundo fallback consecutivo se ofrece escalar a asesor (evita bucles de "no entendí").
Usuario abandona un flujo a mitad de camino	Comando universal "menú" / "cancelar" disponible en cualquier nodo; descarta entidades parcialmente capturadas y regresa al menú principal.
Sesión inactiva por timeout	El contexto parcial se conserva por un tiempo definido (ej. 10 min) y se envía un mensaje de reenganche antes de cerrar la sesión.
Múltiples fallos de autenticación	Tras 3 intentos fallidos de cédula/número de cuenta, se bloquea la sesión temporalmente y se registra el evento como posible intento de acceso indebido.
Consulta repetida en la misma sesión	El bot responde con el dato ya obtenido en lugar de re-consultar el webhook (evita llamadas redundantes).

4.2 Estructura de prompts de sistema

La lógica del agente se define en tres bloques: **reglas y alcance**, **tono**, y **extracción de entidades**. Este prompt se configura como *system prompt* en la plataforma de orquestación (p. ej. Voiceflow/Typebot con LLM, o un nodo de OpenAI dentro de n8n).

[ROL]

Eres InvBot, el asistente conversacional oficial de Andes Valores S.A. (casa de bolsa).

Atiendes exclusivamente a clientes con cuenta de inversión activa.

[ALCANCE Y REGLAS - NO NEGOCIABLES]

1. NO das asesoría de inversión (qué comprar, vender o cuándo). Si el usuario lo solicita,
responde: "Esa decisión requiere el criterio de un asesor certificado" y ofrece
escalar a un asesor humano.
2. NO confirmas ni ejecutas órdenes de compra/venta reales; solo consultas su estado.
3. NO revelas información de otro cliente distinto al autenticado en la conversación.
4. Si el usuario no ha sido identificado (número de cuenta o cédula validados), no debes entregar datos de saldo, órdenes ni movimientos.
5. Toda tarifa, tiempo o política que menciones debe basarse ÚNICAMENTE en los documentos
de referencia cargados (tabla de comisiones, políticas de retiro, calendario bursátil).
Si no tienes el dato, indícalo y ofrece escalar, nunca inventes cifras.
6. Si detectas una queja formal, fraude sospechado o solicitud legal, escala de inmediato
a un asesor humano sin intentar resolverlo tú mismo.
7. Cada incidencia reportada debe generar un ticket con: nombre, correo, teléfono,
motivo de consulta y, si aplica, número de cuenta u orden.

[TONO]

- Profesional, claro y cercano; evita jerga financiera innecesaria.
- Frases cortas. Confirma antes de ejecutar una acción que modifique datos (crear ticket).
- Ante errores o demoras del sistema, discúlpate brevemente y ofrece alternativa (ticket o asesor).
- Nunca uses lenguaje que sugiera garantía de rentabilidad o resultados de inversión.

[EXTRACCIÓN DE ENTIDADES]

Cuando el usuario reporte una incidencia o solicite escalamiento, extrae y valida:

- nombre_completo (string, obligatorio)
- correo (string, formato email válido, obligatorio)
- telefono (string, formato +593XXXXXXXXX o 10 dígitos, obligatorio)
- motivo_consulta (enum: ["orden_no_ejecutada", "error_plataforma", "reclamo_cobro", "consulta_dividendo", "otro"], obligatorio)
- numero_cuenta (string, opcional si ya fue autenticado en la sesión)
- numero_orden (string, opcional, solo si motivo_consulta =

```
"orden_no_ejecutada")
```

Si falta un dato obligatorio, pregúntalo de forma individual (no todos a la vez).

Al completar la extracción, resume los datos y pide confirmación explícita ("¿Confirma que estos datos son correctos?") antes de enviarlos al webhook de creación de ticket.

[FORMATO DE SALIDA PARA INTEGRACIÓN]

Cuando completes una extracción de entidades, además de tu respuesta conversacional, genera internamente el objeto JSON correspondiente (ver sección 5.2) para ser enviado al flujo de automatización.

5. Arquitectura e integración de datos (Fase 3)

5.1 Modelo de base de datos (PostgreSQL / Supabase)

```
-- Extensión para UUID
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Tabla de clientes
CREATE TABLE clientes (
  id_cliente      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nombre_completo VARCHAR(150) NOT NULL,
  correo          VARCHAR(150) NOT NULL UNIQUE,
  telefono        VARCHAR(20)  NOT NULL,
  cedula          VARCHAR(20)  NOT NULL UNIQUE,
  perfil_riesgo   VARCHAR(20)  NOT NULL DEFAULT 'moderado'
  CHECK (perfil_riesgo IN
('conservador','moderado','agresivo')),
  fecha_registro  TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- Tabla de cuentas de inversión
CREATE TABLE cuentas (
  id_cuenta       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  id_cliente       UUID NOT NULL REFERENCES clientes(id_cliente) ON DELETE
  CASCADE,
  numero_cuenta   VARCHAR(20) NOT NULL UNIQUE,
  saldo_disponible NUMERIC(14,2) NOT NULL DEFAULT 0 CHECK (saldo_disponible
  >= 0),
  saldo_custodia  NUMERIC(14,2) NOT NULL DEFAULT 0 CHECK (saldo_custodia >=
  0),
  estado_cuenta   VARCHAR(15) NOT NULL DEFAULT 'activa'
```

```

        CHECK (estado_cuenta IN
('activa','suspendida','cerrada')),
        fecha_apertura    TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Catálogo de instrumentos negociables
CREATE TABLE catalogo_instrumentos (
    id_instrumento        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    codigo_nemonico        VARCHAR(15) NOT NULL UNIQUE,
    nombre                 VARCHAR(150) NOT NULL,
    tipo_instrumento       VARCHAR(20) NOT NULL
        CHECK (tipo_instrumento IN
('accion','bono','obligacion','papel_comercial')),
    mercado                VARCHAR(10) NOT NULL CHECK (mercado IN ('BVQ','BVG'))
);

-- Tabla de órdenes
CREATE TABLE ordenes (
    id_orden              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    id_cuenta              UUID NOT NULL REFERENCES cuentas(id_cuenta) ON DELETE
CASCADE,
    id_instrumento        UUID NOT NULL REFERENCES
catalogo_instrumentos(id_instrumento),
    tipo_orden             VARCHAR(10) NOT NULL CHECK (tipo_orden IN
('compra','venta')),
    cantidad              INTEGER NOT NULL CHECK (cantidad > 0),
    precio_limite          NUMERIC(14,4),
    estado_orden           VARCHAR(15) NOT NULL DEFAULT 'pendiente'
        CHECK (estado_orden IN
('pendiente','ejecutada','cancelada','rechazada')),
    fecha_creacion         TIMESTAMPTZ NOT NULL DEFAULT now(),
    fecha_ejecucion        TIMESTAMPTZ
);

-- Tabla de tickets de incidencias (generados por el chatbot)
CREATE TABLE tickets (
    id_ticket              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    id_cliente             UUID NOT NULL REFERENCES clientes(id_cliente) ON DELETE
CASCADE,
    numero_cuenta          VARCHAR(20),
    numero_orden           VARCHAR(20),
    motivo_consulta        VARCHAR(30) NOT NULL
        CHECK (motivo_consulta IN
('orden_no_ejecutada','error_plataforma',
'reclamo_cobro','consulta_dividendo','otro')),
    descripcion            TEXT,
    estado_solicitud       VARCHAR(15) NOT NULL DEFAULT 'abierto'
        CHECK (estado_solicitud IN
('abierto','en_proceso','resuelto','cerrado')),

```

```

    canal_origen      VARCHAR(20) NOT NULL DEFAULT 'chatbot',
    idempotency_key    VARCHAR(60) UNIQUE,
    fecha_registro     TIMESTAMPTZ NOT NULL DEFAULT now(),
    fecha_actualizacion TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Trigger: actualizar fecha_actualizacion automáticamente en cada UPDATE
CREATE OR REPLACE FUNCTION fn_actualizar_fecha()
RETURNS TRIGGER AS $$
BEGIN
    NEW.fecha_actualizacion = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_tickets_actualizar_fecha
BEFORE UPDATE ON tickets
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_fecha();

-- Registro de cada interacción del chatbot (auditoría / trazabilidad)
CREATE TABLE interacciones_chatbot (
    id_interaccion     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    id_cliente          UUID REFERENCES clientes(id_cliente),
    intent_detectado    VARCHAR(50) NOT NULL,
    mensaje_usuario     TEXT NOT NULL,
    respuesta_bot       TEXT NOT NULL,
    id_ticket           UUID REFERENCES tickets(id_ticket),
    fecha_interaccion   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Índices de apoyo
CREATE INDEX idx_ordenes_cuenta ON ordenes(id_cuenta);
CREATE INDEX idx_tickets_cliente ON tickets(id_cliente);
CREATE INDEX idx_tickets_estado ON tickets(estado_solicitud);

```

Justificación de diseño:

- `UUID` como llave primaria evita colisiones al integrarse con sistemas externos (n8n/Make) que generan IDs antes de insertar.
- Los `CHECK` constraints (`estado_orden`, `estado_solicitud`, `perfil_riesgo`) reemplazan tipos `ENUM` para mantener compatibilidad directa con Supabase.
- `interacciones_chatbot` permite trazabilidad completa de la conversación para auditoría (requisito de "Sustentación Técnica" de la rúbrica).

- Separar `cuentas` de `clientes` permite que un cliente tenga más de una cuenta (ej. cuenta propia y cuenta mancomunada).
- `idempotency_key` en `tickets` evita crear tickets duplicados si el webhook se reintenta por timeout de red — el `INSERT` usa `ON CONFLICT (idempotency_key) DO NOTHING` y devuelve el ticket ya existente.
- El trigger `trg_tickets_actualizar_fecha` garantiza que `fecha_actualizacion` sea siempre confiable, clave para reportar SLA de forma consistente.

Sincronización en tiempo real

Al usar Supabase, la tabla `tickets` puede exponerse mediante Supabase Realtime (canal `postgres_changes`), de modo que un panel interno del asesor reciba el nuevo ticket por WebSocket en el instante en que se inserta, sin necesidad de refrescar ni hacer polling. Esto es lo que se muestra en vivo durante la demo del video: el ticket aparece en la base de datos en el mismo segundo en que el bot confirma el número al usuario.

5.2 Flujo de automatización (Webhooks + n8n)

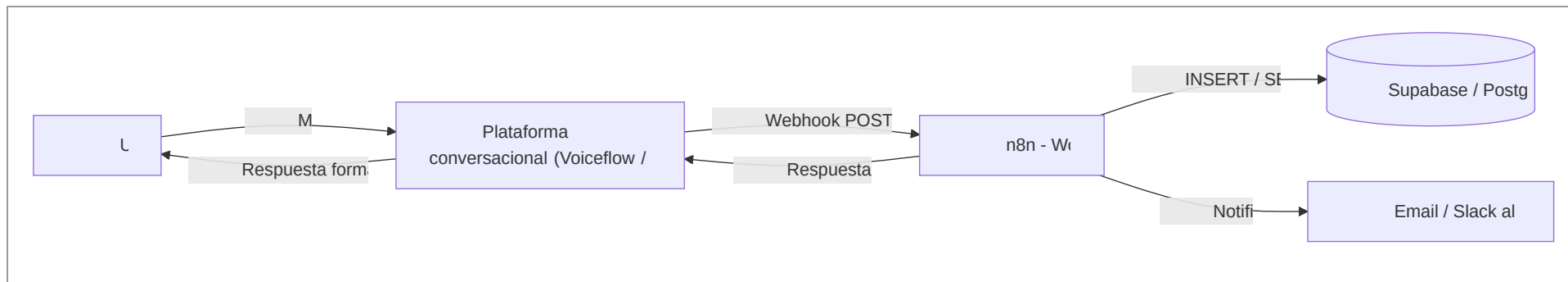


Figura 2. Arquitectura de integración entre la plataforma conversacional, el flujo de automatización (n8n) y la base de datos (Supabase/PostgreSQL).

```

flowchart LR
    U[Usuario] -->|Mensaje| P[Plataforma conversacional<br/>Voiceflow / Typebot]
    P -->|Webhook POST JSON| N[n8n - Workflow]
    N -->|INSERT / SELECT| S[(Supabase / PostgreSQL)]
    N -->|Notificación| E[Email / Slack al asesor]
    N -->|Respuesta JSON| P
    P -->|Respuesta formateada| U

```

Código fuente del diagrama en sintaxis Mermaid.js correspondiente a la Figura 2.

Webhook 1 — Consulta de estado de orden (GET lógico vía POST a n8n)

Payload de entrada (plataforma → n8n):

```

{
  "evento": "consultar_orden",
  "numero_cuenta": "AV-00234",
  "numero_orden": "ORD-88213",
  "session_id": "vf_5f2a9c",
  "timestamp": "2026-08-29T14:32:10-05:00"
}

```

Nodo n8n: Webhook → Postgres (Query) → Set (formatea respuesta) → Respond to Webhook.

Respuesta de n8n → plataforma:

```

{
  "estado": "ok",
  "orden": {
    "numero_orden": "ORD-88213",
    "instrumento": "BANCO PICHINCHA C.A.",
    "tipo_orden": "compra",
    "cantidad": 150,
    "estado_orden": "ejecutada",
    "fecha_ejecucion": "2026-08-29T13:05:00-05:00"
  },
  "mensaje_sugerido": "Tu orden ORD-88213 de compra de 150 acciones fue ejecutada el 29/08/2026 a las 13:05."
}

```

Webhook 2 — Creación de ticket de incidencia

Payload de entrada (generado tras la extracción de entidades del prompt, sección 4.2):

```
{
  "evento": "crear_ticket",
  "nombre_completo": "María Fernanda López",
  "correo": "mflopez@correo.com",
  "telefono": "+593987654321",
  "numero_cuenta": "AV-00234",
  "numero_orden": "ORD-88213",
  "motivo_consulta": "orden_no_ejecutada",
  "descripcion": "El cliente indica que la orden aparece pendiente hace 3 días.",
  "canal_origen": "chatbot",
  "session_id": "vf_5f2a9c",
  "idempotency_key": "vf_5f2a9c-crear_ticket-1756483930"
}
```

Lógica del nodo n8n:

1. **Webhook** recibe el payload.
2. **Function/Set** valida formato de correo y teléfono (regex) — si falla, responde **estado: "error_validacion"** y la plataforma vuelve a pedir el dato.
3. **Postgres (Insert)** en tabla **tickets** usando **ON CONFLICT (idempotency_key) DO NOTHING RETURNING *** (si el cliente no existe aún en **clientes**, un nodo previo **Postgres (Upsert)** lo crea). Si el **INSERT** no retorna filas, un nodo **Postgres (Query)** recupera el ticket ya creado en vez de duplicarlo.
4. **IF** — si **motivo_consulta** es **reclamo_cobro** u **orden_no_ejecutada**, se dispara notificación inmediata.
5. **Send Email / Slack** al asesor con los datos del ticket.
6. **Respond to Webhook** retorna el número de ticket generado.

Respuesta de n8n → plataforma:

```
{
  "estado": "ok",
  "id_ticket": "6f1c2e40-9b3a-4e21-8e2a-3d1a2f9b7c10",
  "numero_ticket_legible": "TCK-2026-0417",
  "sla_estimado_horas": 24,
  "mensaje_sugerido": "Listo, María Fernanda. Registré tu solicitud con el ticket TCK-2026-0417. Un asesor te contactará en un máximo de 24 horas."
}
```

Manejo de errores

Si Postgres (Insert) falla (timeout o violación de constraint), n8n captura el error con un nodo Error Trigger, responde a la plataforma con {"estado":

"error_sistema"} y la plataforma muestra: "Tuvimos un problema técnico al registrar tu solicitud. Por favor intenta nuevamente o escribe 'asesor' para hablar con una persona."

Esto evita pérdida silenciosa de datos, requisito explícito del criterio "Integración de Datos" de la rúbrica.

6. Decisiones de arquitectura y alternativas evaluadas

Decisión	Opción elegida	Alternativas consideradas	Justificación
Plataforma conversacional	Voiceflow / Typebot	Dialogflow CX, Botpress	No-code, integración nativa de webhooks, curva de aprendizaje corta para un proyecto académico con plazo definido.
Base de datos	PostgreSQL (Supabase)	Airtable, Google Sheets	Se requieren relaciones (FK), CHECK constraints y transacciones ACID; Airtable/Sheets no garantizan integridad referencial.
Orquestación de automatización	n8n	Make, Zapier, Power Automate	n8n permite exportar el flujo completo como JSON versionable (reproducibilidad) y ejecutar lógica condicional compleja sin límite de pasos.
Identificación de cliente	Cédula + número de cuenta	Solo número de cuenta	Un solo dato es fácil de adivinar/filtrar; combinar dos reduce el riesgo sin requerir

			infraestructura de 2FA, fuera del alcance del prototipo.
Registro de incidencias	Tabla tickets con estados	Enviar solo un correo al asesor	Un correo no es consultable ni auditable; una tabla con estados permite medir SLA y dar seguimiento verificable.

7. Seguridad, pruebas y estrategia de despliegue

7.1 Seguridad

- **Autenticación mínima:** validación de número de cuenta + cédula antes de exponer datos de saldo u órdenes.
- **Variables sensibles:** credenciales de Supabase y tokens de n8n como variables de entorno, nunca en el prompt ni en el repositorio.
- **Principio de mínimo privilegio:** el usuario de PostgreSQL que usa n8n solo tiene permisos `INSERT / SELECT / UPDATE` sobre las tablas necesarias.
- **No persistencia de datos sensibles en el prompt:** el LLM no almacena el número de cédula completo más allá de lo necesario para la sesión activa.

7.2 Pruebas

Casos de prueba funcionales mínimos a documentar con evidencia (captura o log) en el repositorio:

1. Flujo feliz de consulta de saldo autenticado.
2. Intento de consulta sin autenticación (debe bloquear y no filtrar datos).
3. Creación de ticket con datos incompletos (debe re-preguntar el campo faltante, uno a la vez).
4. Reenvío del mismo webhook dos veces → debe crear un solo ticket gracias a `idempotency_key`.
5. Simulación de caída del webhook / base de datos → debe mostrar el mensaje de error controlado.

6. Tres intentos fallidos de autenticación → debe bloquear la sesión y registrar el evento.

7.3 Estrategia de despliegue

- **Plataforma conversacional:** publicada mediante el enlace de despliegue nativo de Voiceflow/Typebot, accesible sin instalación.
- **Automatización:** instancia de n8n Cloud (o self-hosted vía Docker) con el workflow activo y el webhook en modo producción.
- **Base de datos:** proyecto de Supabase en la nube; el script `schema.sql` se ejecuta una sola vez desde el SQL Editor para recrear el entorno completo.
- **Control de versiones:** el workflow de n8n se exporta periódicamente como `.json` y el `schema.sql` se versiona junto al informe, permitiendo reconstruir el entorno desde cero.

8. Entregables del proyecto

Esta sección enlaza explícitamente los cuatro entregables exigidos por la actividad, para trazabilidad del informe:

1. Informe técnico (este documento): justificación, entorno profesional, arquitectura, esquemas de base de datos y diagramación del flujo conversacional.

2. Prototipo funcional: InvBot, con el flujo conversacional completo (identificación, consultas, creación de tickets) conectado a un proyecto real de Supabase/PostgreSQL: la autenticación consulta las tablas `clientes`, `cuentas` y `ordenes` en vivo, y cada ticket reportado se inserta en la tabla `tickets` real y aparece de inmediato en el panel gracias a Supabase Realtime. Enlace entregado junto con este informe.

3. Repositorio / configuración técnica: repositorio público en GitHub (github.com/abelsanchez1925/invbot-andes-valores), que incluye

`supabase_setup.sql` (esquema, datos de prueba, políticas RLS y activación de Realtime), el archivo del prototipo funcional, el informe técnico y el material de apoyo para la sustentación.

4. Video de sustentación: grabado en Blackboard Video Studio siguiendo `guion_video.md`, con identificación, demo en vivo y defensa de código.

9. Conclusiones y reflexión final

InvBot demuestra que una arquitectura conversacional simple (menú guiado + extracción de entidades) combinada con un flujo de automatización sin código (n8n) y una base de datos relacional puede resolver un problema real de atención al cliente en el sector bursátil: reducir tiempos de espera, garantizar trazabilidad de incidencias y liberar a los asesores humanos de consultas repetitivas para que se enfoquen en asesoría de valor.

El mayor reto de diseño fue el prompt de sistema: encontrar el balance entre un tono cercano y reglas estrictas para que el bot nunca dé asesoría de inversión ni exponga datos de otro cliente. La lógica de manejo de casos límite (fallback, timeout, reintentos) resultó tan importante para la calidad percibida del asistente como el flujo "feliz" principal, y el uso de `idempotency_key` fue la decisión técnica que más fortaleció la confiabilidad de la integración de datos.

10. Referencias

Hall, E. (2018). *Conversational design*. A Book Apart.

n8n GmbH. (2026). *n8n documentation*. <https://docs.n8n.io>

Open Worldwide Application Security Project. (2023). *OWASP top 10:2023*.

<https://owasp.org/www-project-top-ten/>

PostgreSQL Global Development Group. (2026). *PostgreSQL 16 documentation*.

<https://www.postgresql.org/docs/>

Shevat, A. (2017). *Designing bots: Creating conversational experiences*. O'Reilly Media.

Supabase Inc. (2026). *Supabase documentation*. <https://supabase.com/docs>

11. Anexos

Los siguientes materiales complementan este informe y se entregan junto con él:

- Anexo A — Esquema SQL completo: sección 5.1 de este informe (archivo `supabase_setup.sql` en el repositorio).
- Anexo B — Payloads de ejemplo (JSON): sección 5.2 de este informe.
- Anexo C — Diapositivas de apoyo para la sustentación: `diapositivas_invbot.pptx`.
- Anexo D — Guion de sustentación en video: documento `guion_video.md`.
- Anexo E — Prototipo funcional interactivo: enlace entregado junto con este informe.

Documento generado para el proyecto académico "Diseño e implementación de un asistente conversacional inteligente aplicado al entorno profesional".